SIMATS ENGINEERING
ASSESSMENT TOOL 1

COURSE NAME: Database Management Systems
COURSE CODE: CSA05

COURSE OUTCOME COVERED

CO2: *Apply the relational model, relational algebra, SQL query structures, and views to solve database querying and data manipulation problems.* (BL3, BL6)

Activity Tool : Analytical Problem Solving (High)
Assessment Tool Weightage: 35%

Dr.G.Ayyappan

QUESTIONS			Total Marks: 50
Q.No	**Question**	**Bloom's Level**	**Marks**
1	Apply relational algebra operations (σ, π, ∪, -, ×, ⋈) on the given Employee and Department relations to retrieve required records.	BL3 – Apply	**5**
2	Write SQL DDL statements to create STUDENT(SID, Name, DOB, DeptID) and DEPARTMENT(DeptID, DeptName) tables with all appropriate constraints.	BL3 – Apply	**5**
3	Formulate SQL queries using GROUP BY, HAVING, and aggregate functions (COUNT, SUM, AVG, MAX, MIN) on a given Sales dataset.	BL3 – Apply	**5**
4	Write a correlated sub-query to find employees earning more than the average salary of their own department.	BL3 – Apply	**5**
5	Apply all types of JOIN operations (INNER, LEFT OUTER, RIGHT OUTER, FULL OUTER, CROSS) on Employee and Project tables.	BL3 – Apply	**5**
6	Create a view 'DeptSalaryView' that displays department name, total employees, and average salary. Write a query on this view.	BL6 – Create	**5**
7	Construct a relational algebra expression for the following: Find names of students who are enrolled in all courses offered.	BL6 – Create	**5**

8	Write SQL DML statements (INSERT, UPDATE, DELETE) with integrity constraint enforcement for a given scenario.	BL3 – Apply	**5**
9	Apply tuple relational calculus to express a query: Find all employees who work in the 'IT' department and earn more than 50000.	BL3 – Apply	**5**
10	Design a relational schema for an Airline Reservation System and write 5 SQL queries demonstrating joins, sub-queries, and views.	BL6 – Create	**5**

Code of Conduct:

*I **KESHAVARAM L /192511253** (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.*

Signature of the student:

RUBRICS FOR CALCULATION:

Numerical Problems					
Criteria	**Weightage**	**Level 4 (Exemplary)**	**Level 3 (Proficient)**	**Level 2 (Developing)**	**Level 1 (Beginning)**
Problem Understanding	20%	Clearly interprets problem, identifies all parameters and requirements accurately	Understands problem with minor gaps	Partial understanding; misses key elements	Misinterprets or unable to understand problem

Method Selection	20%	Selects most appropriate method/formula with strong justification	Selects correct method with minor errors	Inappropriate or partially correct method	Unable to select method
Solution Process	25%	Logical, systematic steps with correct approach throughout	Mostly correct steps with minor mistakes	Disorganized steps; multiple errors	No clear process
Accuracy of Calculation	20%	All calculations accurate; correct final answer	Minor calculation errors	Frequent errors; incorrect final answer	No valid calculations
Interpretation of Results	15%	Clearly interprets results with units and engineering relevance	Basic interpretation with minor omissions	Limited or unclear interpretation	No interpretation

1. RELATIONAL ALGEBRA OPERATIONS ON EMPLOYEE AND DEPARTMENT

PROBLEM:

An organization stores employee details in EMPLOYEE and department details in DEPARTMENT. We need to retrieve specific employee and department information using relational algebra.

Given:

- EMPLOYEE(EID, EName, Salary, DeptID)
- DEPARTMENT(DeptID, DeptName)

Analysis

Different relational algebra operators are required for different types of data retrieval:

- **σ (Selection)** → selects rows.
- **π (Projection)** → selects columns.
- **∪ (Union)** → combines compatible relations.
- **− (Difference)** → finds records present in one relation but not another.
- **× (Cartesian Product)** → combines every row of two relations.
- **⋈ (Join)** → combines related records using a common attribute.

Solution

Employees earning more than 50,000:

Display employee names and salaries:

Employees from Department 10 or 20:

Employees not belonging to Department 10:

Cartesian product:

Employee and department details:

Result

Required employee and department records are retrieved using appropriate relational algebra operations.

2. STUDENT AND DEPARTMENT DATABASE USING DDL

PROBLEM

A college needs a database to store student information and their department details while preventing invalid or duplicate data.

Analysis

The database requires:

- Unique student identification.
- Unique department identification.
- Mandatory student name and date of birth.
- A relationship between students and departments.

Solution

```
CREATE TABLE DEPARTMENT (
   DeptID INT PRIMARY KEY,
   DeptName VARCHAR(50) NOT NULL UNIQUE
);

CREATE TABLE STUDENT (
   SID INT PRIMARY KEY,
   Name VARCHAR(50) NOT NULL,
   DOB DATE NOT NULL,
   DeptID INT,
   FOREIGN KEY (DeptID) REFERENCES DEPARTMENT(DeptID)
);
```

Result

The tables are created successfully with primary key, foreign key, NOT NULL, and UNIQUE constraints to maintain data integrity.

3. SALES ANALYSIS USING GROUP BY AND AGGREGATE FUNCTIONS

PROBLEM

A company wants to analyze its sales according to product categories and calculate total, average, maximum, minimum, and number of sales.

Given

SALES(SaleID, Product, Category, Quantity, Amount)

Analysis

Since calculations are required for each category, GROUP BY is used. Aggregate functions calculate the required values.

Solution

```
SELECT Category, COUNT(*) AS TotalSales
FROM SALES
GROUP BY Category;
SELECT Category, SUM(Amount) AS TotalAmount
FROM SALES
GROUP BY Category;
SELECT Category, AVG(Amount) AS AverageAmount
FROM SALES
GROUP BY Category;
SELECT Category, MAX(Amount) AS MaximumAmount,
    MIN(Amount) AS MinimumAmount
FROM SALES
GROUP BY Category;
```

To find categories with total sales above 100000:

```
SELECT Category, SUM(Amount) AS TotalAmount
FROM SALES
GROUP BY Category
HAVING SUM(Amount) > 100000;
```

Result

The sales data is grouped by category and analyzed using COUNT, SUM, AVG, MAX, MIN, and HAVING.

4. EMPLOYEES EARNING MORE THAN THEIR DEPARTMENT AVERAGE

PROBLEM

A company wants to identify employees whose salary is greater than the average salary of their **own department**.

Analysis

The average salary must be calculated separately for each employee's department. Therefore, the inner query must refer to the department of the current employee. This makes it a **correlated sub-query**.

Solution

```
SELECT E1.EID, E1.EName, E1.Salary, E1.DeptID
FROM EMPLOYEE E1
WHERE E1.Salary > (
   SELECT AVG(E2.Salary)
   FROM EMPLOYEE E2
   WHERE E2.DeptID = E1.DeptID
);
```

Result

The query identifies employees earning more than the average salary of their respective departments.

5. JOIN OPERATIONS ON EMPLOYEE AND PROJECT

PROBLEM

A company wants to analyze the relationship between employees and the projects assigned to them.

Given

EMPLOYEE(EID, EName, DeptID)

PROJECT(PID, PName, EID)

Analysis

Different joins provide different information:

- **INNER JOIN** → matching employees and projects.
- **LEFT JOIN** → all employees, including those without projects.
- **RIGHT JOIN** → all projects, including those without matching employees.
- **FULL JOIN** → all records from both tables.
- **CROSS JOIN** → every possible employee-project combination.

Solution

INNER JOIN

```
SELECT E.EName, P.PName
FROM EMPLOYEE E
INNER JOIN PROJECT P ON E.EID = P.EID;
```

LEFT JOIN

```
SELECT E.EName, P.PName
FROM EMPLOYEE E
LEFT JOIN PROJECT P ON E.EID = P.EID;
```

RIGHT JOIN

```
SELECT E.EName, P.PName
FROM EMPLOYEE E
RIGHT JOIN PROJECT P ON E.EID = P.EID;
```

FULL OUTER JOIN

```
SELECT E.EName, P.PName
FROM EMPLOYEE E
FULL OUTER JOIN PROJECT P ON E.EID = P.EID;
```

CROSS JOIN

```
SELECT E.EName, P.PName
```

```
FROM EMPLOYEE E
CROSS JOIN PROJECT P;
```

Result

Different types of joins provide different views of employee-project relationships.

6. DEPARTMENT SALARY VIEW

PROBLEM

Management wants a reusable report showing each department's total number of employees and average salary.

Analysis

The same calculation may be required repeatedly. Creating a **view** avoids rewriting the complete query.

Solution

```
CREATE VIEW DeptSalaryView AS
SELECT D.DeptName,
    COUNT(E.EID) AS TotalEmployees,
    AVG(E.Salary) AS AverageSalary
FROM DEPARTMENT D
LEFT JOIN EMPLOYEE E
ON D.DeptID = E.DeptID
GROUP BY D.DeptID, D.DeptName;
```

To retrieve the report:

```
SELECT * FROM DeptSalaryView;
```

To find departments with average salary above 50000:

```
SELECT *
FROM DeptSalaryView
WHERE AverageSalary > 50000;
```

Result

The view provides a simplified and reusable department salary report.

7. STUDENTS ENROLLED IN ALL COURSES

PROBLEM

A university wants to find students who have enrolled in **every course offered**.

Given

- STUDENT(SID, SName)
- ENROLL(SID, CID)
- COURSE(CID, CName)

Analysis

The requirement "enrolled in **all courses**" indicates the use of the **division operator (÷)** in relational algebra.

First, obtain student-course enrollment pairs:

Then obtain all available courses:

Apply division:

Finally, join with STUDENT to obtain names.

Solution

Result

The expression returns the names of students enrolled in every course offered by the university.

8. DML WITH INTEGRITY CONSTRAINTS

PROBLEM

An organization needs to add, modify, and remove employee records while ensuring that invalid department references are not accepted.

Analysis

DML operations are:

- INSERT → add records.
- UPDATE → modify records.

- DELETE → remove records.

The foreign key ensures that an employee cannot refer to a non-existing department.

Solution

Insert department:

```
INSERT INTO DEPARTMENT
VALUES (10, 'IT');
```

Insert employee:

```
INSERT INTO EMPLOYEE
VALUES (101, 'Arun', 55000, 10);
```

Update salary:

```
UPDATE EMPLOYEE
SET Salary = 60000
WHERE EID = 101;
```

Delete employee:

```
DELETE FROM EMPLOYEE
WHERE EID = 101;
```

Result

Employee records are inserted, updated, and deleted while database integrity is maintained through constraints.

9. TUPLE RELATIONAL CALCULUS

PROBLEM

A company wants to find employees who belong to the **IT department** and earn more than **50000**.

Given

- EMPLOYEE(EID, EName, Salary, DeptID)
- DEPARTMENT(DeptID, DeptName)

Analysis

Two conditions must be satisfied:

1. Employee belongs to IT department.
2. Employee salary is greater than 50000.

Tuple relational calculus expresses **what data is required**, rather than how to retrieve it.

Solution

Result

The expression returns the names of employees working in IT whose salary exceeds 50000.

10. AIRLINE RESERVATION SYSTEM

PROBLEM

An airline needs a database to manage passengers, flights, bookings, and payments.

Analysis

The system requires:

- Passenger details.
- Flight details.
- Booking information.
- Payment information.
- Relationships between passengers and flights.

Relational Schema

PASSENGER(PassengerID, PassengerName, Phone, Email)

AIRPORT(AirportID, AirportName, City)

FLIGHT(FlightID, FlightNo, Source, Destination,
DepartureTime, ArrivalTime)

BOOKING(BookingID, PassengerID, FlightID,
BookingDate, SeatNo)

PAYMENT(PaymentID, BookingID, Amount,
PaymentDate, Status)

Query 1 – Passenger and Booking

```
SELECT P.PassengerName, B.BookingID, B.SeatNo
FROM PASSENGER P
JOIN BOOKING B
ON P.PassengerID = B.PassengerID;
```

Query 2 – Passenger and Flight

```
SELECT P.PassengerName, F.FlightNo, F.Destination
FROM PASSENGER P
JOIN BOOKING B
ON P.PassengerID = B.PassengerID
JOIN FLIGHT F
ON B.FlightID = F.FlightID;
```

Query 3 – Sub-query

Find passengers who booked flights to Chennai.

```
SELECT PassengerName
FROM PASSENGER
WHERE PassengerID IN (
   SELECT B.PassengerID
   FROM BOOKING B
   JOIN FLIGHT F
   ON B.FlightID = F.FlightID
   WHERE F.Destination = 'Chennai'
);
```

Query 4 – View

```
CREATE VIEW FlightBookingView AS
SELECT F.FlightNo,
       F.Source,
       F.Destination,
       COUNT(B.BookingID) AS TotalBookings
FROM FLIGHT F
LEFT JOIN BOOKING B
ON F.FlightID = B.FlightID
GROUP BY F.FlightID, F.FlightNo, F.Source, F.Destination;
```

View query:

```
SELECT *
FROM FlightBookingView;
```

Query 5 – Sub-query with Aggregate

Find flights having more bookings than the average number of bookings.

```
SELECT FlightID, COUNT(*) AS TotalBookings
FROM BOOKING
GROUP BY FlightID
HAVING COUNT(*) > (
   SELECT AVG(Cnt)
   FROM (
      SELECT COUNT(*) AS Cnt
      FROM BOOKING
      GROUP BY FlightID
   ) X
);
```

Result

The Airline Reservation System successfully manages passenger, flight, booking, and payment information using **joins, sub-queries, aggregate functions, and views**.